

Digit Symbol Substitution Test

DSST Project — Developer & Student Guide

How the task works, how to configure it, and how to make it your own

wecare_dsst | Phaser.js 3 + jsPsych 7.1.2 | Mobile-first browser app | No build step required

What changed in this version

- Data collection is now fully wired up — `sendData.js` is imported and called after each run
- Feedback is shown after every response: stimulus box turns green (correct) or red (incorrect) in both baseline and practice scenes
- `FeedbackHandler` gameobject added (`src/gameobjects/feedbackHandler.js`)
- `debug: false` is now the default in `run.js`; `debug: true` remains in `baseline.js` `TrialHandler`
- End screen (`end_screen`) is now shown after the run completes, before data is uploaded
- Language selection via `?lang=nl|en` URL parameter; `ENG_text.json` added as English locale
- URL parameters now support: `participant`, `lang`, `beepnum`, `testday` — all read in `index.html`
- `boot.js` now collects device info (FPS, pixel ratio, OS, language) and logs session metadata via `jsPsych.data.addProperties()`
- Canvas now scales by DPR (width: $360 * \text{assetSize}$, height: $640 * \text{assetSize}$) instead of fixed 360 x 640
- Symbol set time windows updated: now 8-11, 12-13, 14-15, 16-17, 18-20 (shifted from earlier ranges)
- `TrialHandler` now writes `trial.index`, `trial.UTCdate`, `trial.type`, `stim.type`, `stim.onset` to `jsPsych` data each trial
- `practice.js` uses `interStimulusInterval: 500` and also shows response feedback tint
- `data/` folder now contains real session CSV files from Pavlovica

1. What Is the DSST?

The Digit Symbol Substitution Test (DSST) is a classic neuropsychological measure of processing speed, attention, and working memory. In the traditional paper version, participants work through a grid of symbols and fill in the matching digit for each one, consulting a reference key. The faster and more accurately they complete the grid, the better the score.

The `wecare_dsst` project is a fully self-contained, browser-based version of this task built for mobile phones. No installation is required — it runs in a browser tab. It is built on two open-source libraries bundled as static files, so there is no package manager or build step:

- Phaser.js (v3, included as `src/libs/phaser.min.js`) — a 2D game framework that renders the task visuals and handles touch/click input.
- jsPsych (v7.1.2, included as `src/libs/jspsych-7.1.2/jspsych.js`) — used for data collection and the Pavlovia upload plugin.

The complete participant experience follows this sequence:

- Boot — silently loads all assets (textures, fonts, text), detects device resolution, collects device metadata, and logs session parameters via jsPsych.
- Border — a decorative frame overlay that stays visible on top of all subsequent scenes.
- Start — splash screen; the participant taps a button to begin.
- DialogBox (start_message) — a transition screen with an introductory message.
- Baseline — a warm-up with digits: a single digit appears at the top, the participant presses the matching button on the numpad. Green/red feedback tint is shown after each response.
- DialogBox (practice_message) — transition message explaining that digits will now be replaced by symbols.
- Practice — a practice round using the practice symbol set ('p'); a symbol appears and the participant finds its digit in the reference key. Green/red feedback tint is shown after each response.
- DialogBox (run_message) — final prompt telling the participant to answer as many as possible in 60 seconds.
- Run — the actual timed DSST: 60 seconds, symbols from a set determined by the current day and hour. After completion, an end screen is displayed and data is uploaded to Pavlovia.

[i] The Border scene is special. Unlike all others, it runs with `active: true`, meaning it launches automatically alongside Boot and stays layered on top of every other scene for the entire session. It is never stopped or replaced.

2. Project Structure

There is no `package.json`, no `npm`, and no build step. All JavaScript files are ES modules loaded directly by the browser. The full folder layout is:

Path	Description
<code>index.html</code>	Entry point — loads Phaser, jsPsych, and <code>src/main.js</code> ; sets global URL parameters (LANG, PARTICIPANT_ID, BEEPNUM, TESTDAY)
<code>assets/css/style.css</code>	CSS variables: colors, fonts, canvas reset
<code>assets/fonts/</code>	Lexend-Regular.ttf and Lexend-Bold.ttf
<code>assets/images/1x/ 2x/ 3x/</code>	border.png at three resolutions

assets/text/NED_text.json	All participant-facing instruction text (Dutch)
assets/text/ENG_text.json	English locale — new in this version
assets/textures/	Sprite atlases (gui + symbols) at 1x, 2x, and 3x
data/	Real session CSV files saved by Pavlovia — new in this version
src/main.js	Phaser config, scene list, orientation lock, DPR helper
src/scenes/boot.js	Loads assets; exposes shared static variables; collects device info into jsPsych data
src/scenes/border.js	Persistent decorative border overlay (active: true)
src/scenes/dialogbox.js	Reusable text popup; uses Message gameobject
src/scenes/start.js	Title/splash screen with start button
src/scenes/baseline.js	Digit warm-up trial; shows green/red tint feedback per response
src/scenes/practice.js	Symbol practice trial; 500 ms ISI; shows green/red tint feedback per response
src/scenes/run.js	Main timed DSST; shows end screen on complete; calls sendData()
src/gameobjects/button.js	Phaser Container: sprite + label, press animation, callback
src/gameobjects/countdown.js	3-2-1 countdown timer; emits 'complete' event
src/gameobjects/feedbackHandler.js	NEW — popup overlay showing correct/incorrect feedback with optional animation
src/gameobjects/gameInstructions.js	Animated instruction overlay using Phaser tween chains
src/gameobjects/gameUI.js	Symbol-key reference grid + 3x3 numpad; records response data to jsPsych
src/gameobjects/message.js	Text box + button for DialogBox scenes
src/gameobjects/sendData.js	Pavlovia upload function — now imported and called in run.js after each session
src/gameobjects/trialHandler.js	Core trial/block engine; writes per-trial data to jsPsych on every trial

To run the project locally, open the folder in VS Code and use the Live Server extension. Alternatively run:

```
python -m http.server 8000
```

then open <http://localhost:8000/index.html>. A local server is required because the browser blocks ES module imports from the file:// protocol.

3. How the Scene System Works

Phaser organises the application as a list of scenes. The full list is defined in `src/main.js` inside the Phaser config object:

```
scene: [Boot, Border, DialogBox, Start, Baseline, Practice, Run]
```

Each scene is a JavaScript class. Phaser calls lifecycle methods on them in order: `preload()` to load assets, then `create()` to set up objects, then `update()` on every frame (not used in this project). Only one scene is active at a time — except for `Border`, which runs persistently on top.

3.1 main.js — Phaser config and helpers

`main.js` configures Phaser and provides two utilities used throughout the project:

- `scaleSize()` — reads `window.devicePixelRatio` and returns 1, 2, or 3. This integer is used to load the matching resolution asset files (`atlas_gui_1x`, `atlas_gui_2x`, etc.) and to scale font sizes.
- `checkOrientation()` — called on load and on every window resize. If the device is landscape (`width > height`), Phaser pauses the current scene and shows a `confirm()` dialog asking the user to rotate back to portrait.

Notable Phaser config changes in this version:

- Canvas dimensions are now dynamic: `width: 360 * assetSize` and `height: 640 * assetSize`. On a 3x device this produces a 1080 x 1920 canvas, giving sharper rendering than the old fixed 360 x 640.
- `backgroundColor` is now set from `Boot.primaryColor`, so the CSS variable propagates into the Phaser config at startup.

NEW Canvas is now `360 * assetSize x 640 * assetSize (DPR-scaled)` instead of a fixed 360 x 640.

3.2 boot.js — global state, asset loading, and device info

The `Boot` scene is the only scene with a `preload()` method. In this version it does three things:

- Loads all shared assets into Phaser's cache (gui atlas, symbol atlas, and text JSON) at the device-appropriate resolution.
- Exposes shared static properties on the `Boot` class itself (`canvasWidth`, `canvasHeight`, `canvasCenterX`, `canvasCenterY`, `assetSize`, `headerFontSize`, `bodyFontSize`, `lightColor`, `darkColor`, `primaryColor`, `secondaryColor`).
- **NEW**: Calls `deviceInfo()` in `create()` to log device metadata and write it as `jsPsych` data properties, so it appears in every row of the exported CSV.

NEW `boot.js` now calls `deviceInfo()` at startup. This logs `fps`, `pixel ratio`, `OS`, `browser language`, `jsPsych version`, `Phaser version`, `participant_id`, `beepnum`, and `testday` — and adds them all to every trial row via `jsPsych.data.addProperties()`.

Language is now also handled in `boot.js preload()`:

```
if (LANG === "nl") { this.load.json("text", "assets/text/NED_text.json"); }
else if (LANG === "en") { this.load.json("text", "assets/text/ENG_text.json"); }
```

`LANG` is set by the `?lang=` URL parameter (default: `"nl"`). See Section 8 for text keys.

CSS variables read by `Boot` (unchanged from previous version):

CSS variable	Default value	Effect in the app
--primary-color	#d4aa55 (gold)	App background, loading bar fill, some button accents
--secondary-color	#213e60 (dark blue)	Progress bar background box
--text-dark-color	#131f2e (near black)	Numbers on the grid, dark button labels
--text-light-color	#f2f2f2 (off-white)	Light text, countdown numbers
--body-font-size	20px	Instruction text size (Lexend-Regular)
--header-font-size	24px	Button labels and large stimulus text (Lexend-Bold)

3.3 border.js — the persistent overlay

The Border scene is unique: it is constructed with `active: true`, which causes Phaser to run it automatically at startup. Its `create()` method loads a `border.png` image that frames the game canvas, then calls `this.scene.bringToTop('border')` to keep it above all other scenes. Because it never stops, the decorative border frame is always visible regardless of which task scene is running. This scene is unchanged in this version.

3.4 dialogbox.js and message.js — transitions

DialogBox is a reusable scene that displays a message and a button between task phases. It is launched with three parameters: `textKey` (which key to look up in the text JSON), `buttonText` (label for the button), and `sceneKey` (which scene to start when the button is pressed).

DialogBox delegates to the Message gameobject, which handles the visual layout: a fade-in, a `text_box` image from the gui atlas, the message text drawn over it, and a Button below. When the button is pressed, the scene fades out and starts the target scene.

***[i]** The instruction text inside DialogBox messages uses `\n` for line breaks. The instruction text used inside GameInstructions (baseline and practice scenes) is split by comma — each comma-separated segment becomes one instruction step.*

4. URL Parameters and Global Variables

NEW This section is new. URL parameters are now the primary way to configure a session.

The following global constants are set in `index.html` from URL query parameters. They are available to all scenes and gameobjects as plain JavaScript globals:

Variable	URL parameter	Default / Purpose
LANG	?lang=nl en	"nl" — selects NED_text.json or ENG_text.json in boot.js

PARTICIPANT_ID	?participant=123	Random UUID — used as filename identifier in Pavlovia CSV export
BEEPNUM	?beepnum=2	1 — EMA beep/notification number for this session
TESTDAY	?testday=3	1 — study test day counter; written to every trial row in the CSV

Example URL to launch a session:

```
https://yourserver/index.html?lang=en&participant=351396&beepnum=2&testday=5
```

5. The Core Engine: TrialHandler

TrialHandler (src/gameobjects/trialHandler.js) manages the entire trial-and-block loop. It is instantiated in each task scene (baseline, practice, run) with a configuration object, and it emits a 'complete' event when all blocks are done.

***new** TrialHandler now writes structured data to jsPsych on every trial: trial.index, trial.UTCdate, trial.type, stim.type, and stim.onset are set via this.scene.data.set() and pushed to jsPsych.data.get() in stopStimulus().*

5.1 Constructor parameters

Parameter	Type	Default	What it controls
scene	Phaser.Scene	required	The scene in which stimuli are rendered
numTrials	number	1	How many trials per block
numBlocks	number	1	How many blocks to run
blockDuration	number (s)	0	Time limit per block in seconds. 0 = ends by trial count
randomize	boolean	true	Whether to shuffle stimuli before each block
randomType	string	'shuffle'	'shuffle' (Fisher-Yates) or 'pick' (random no-consecutive-repeat)
stimulusDuration	number (ms)	0	How long to show each stimulus. 0 = wait for participant response
interStimulusInterval	number (ms)	0	Blank gap between stimuli in milliseconds
stimuli	string[]	required	Array of stimulus keys — plain strings for text, or atlas frame names for images
stimulusType	string	required	'text', 'image', or 'sprite' (sprite not yet implemented)

atlas	string	'guiAtlas'	Texture atlas to use when stimulusType is 'image'
x	number	required	Horizontal canvas position of the stimulus
y	number	required	Vertical canvas position of the stimulus
debug	boolean	false	Logs trial index, stimulus key, and each response to the browser console

5.2 How the trial and block loop works

- `runBlocks()` is called by the scene to start. If `blockDuration > 0`, a Phaser timer is set for that many seconds; when it fires, 'complete' is emitted immediately.
- `runBlocks()` resets `trialIndex` to 0, calls `setPresentationRate()` to build the trial stimulus list, then calls `runTrials()`.
- `runTrials()` checks if `trialIndex >= numTrials`. If so, it increments `blockIndex` and calls `runBlocks()` to start the next block. Otherwise it looks up `trialStimuli[trialIndex]` and calls `setStimulus()` followed by `stopStimulus()`.
- `setStimulus()` adds the stimulus to the scene and records `stim.onset` (timestamp) to the scene data store.
- `stopStimulus()` either waits for `stimulusDuration` milliseconds or listens for the 'playerResponse' event emitted by GameUI. When either fires, it pushes the complete data object to `jsPsych.data` and calls `runTrials()` again.

***[i]** In `run.js`, `blockDuration` is 60 seconds. This means the run ends after 60 seconds regardless of how many of the 180 possible trials were completed. The trial count is effectively a ceiling, not a target.*

5.3 Randomization methods

- 'shuffle': Repeats the stimuli array enough times to fill `numTrials`, then applies Phaser's built-in Fisher-Yates shuffle. Every symbol appears an equal number of times.
- 'pick': Picks one stimulus at a time randomly. It checks that the last two picks differ from the current pick to avoid immediate repetition.
- `randomize: false` — stimuli are used in the exact order of the stimuli array, repeated as needed.

6. Game UI: Symbol Key and Numpad

GameUI (`src/gameobjects/gameUI.js`) builds and manages all visible interactive elements during a task scene. It creates the following scene objects:

- `scene.stimulusBox` — the image box at the top where the current stimulus appears.
- `scene.paper` — a paper-background image at the bottom, behind the symbol key.
- `scene.gridContainer` — a Phaser container holding the grid image, 9 number labels, and 9 symbol images.

- `scene.buttons` — a Phaser container holding 9 Button objects (labels 1-9) in a 3x3 grid.

NEW `recordResponse()` now also writes response data: `response.type` (button label), `response.onset` (timestamp), `response.RT` (reaction time), `stim.key` (correct digit), `response.key` (symbol pressed), and `response.correct` (1/0) — all saved per trial into the jsPsych data pipeline.

6.1 Symbol order in the key

The `symbolOrder` parameter (an array of 9 numbers) controls the mapping of symbols to digit positions in the key grid. `symbolOrder[i]` tells GameUI which digit position corresponds to the *i*-th symbol, allowing counterbalanced layouts across sessions.

In `run.js` the `symbolOrder` is set dynamically by `setStimuli()` based on the day of the week. The `symbolSet` is set based on the hour of the day.

NEW Symbol set time windows have shifted in this version:

Time window	Symbol set (new)	Symbol set (old)
08:00 – 11:59	s01	07:00 – 09:59 (was s01)
12:00 – 13:59	s02	10:00 – 12:59 (was s02)
14:00 – 15:59	s03	13:00 – 15:59 (was s03)
16:00 – 17:59	s04	16:00 – 18:59 (was s04)
18:00 – 20:59	s05	19:00 – 21:59 (was s05)
All other hours	'p' (practice set, fallback)	Same

Symbol order per day of week (unchanged):

Day	symbolOrder (key rotation)
Sunday	[4, 5, 6, 7, 8, 9, 1, 2, 3]
Monday	[6, 7, 8, 9, 1, 2, 3, 4, 5]
Tuesday	[8, 9, 1, 2, 3, 4, 5, 6, 7]
Wednesday	[1, 2, 3, 4, 5, 6, 7, 8, 9]
Thursday	[7, 8, 9, 1, 2, 3, 4, 5, 6]
Friday	[5, 6, 7, 8, 9, 1, 2, 3, 4]
Saturday	[9, 1, 2, 3, 4, 5, 6, 7, 8]

6.2 GameUI methods

Method	What it does
<code>show({ trial: 'baseline' })</code>	Shows stimulus box and numpad; hides symbol key and paper. Background set to dark grey (#4f4d4c).

<code>show({ trial: 'run' })</code>	Shows all elements: stimulus box, numpad, symbol key, and paper background.
<code>hide()</code>	Hides all elements. Background resets to <code>Boot.primaryColor</code> (the CSS <code>--primary-color</code>).
<code>recordResponse(true)</code>	Attaches touch/click listeners to all 9 numpad buttons. Each press records response data to the scene data store and emits 'playerResponse'.

7. Task Scenes in Detail

7.1 baseline.js — digit warm-up

Baseline shows a plain text digit at the top of the screen and asks the participant to press the matching button. No symbol key is shown — only the numpad and the digit stimulus.

Parameter	Value in code	Meaning
<code>numTrials</code>	9	One trial per digit (1–9), shown once each
<code>numBlocks</code>	1	Single block
<code>stimuli</code>	<code>["1", ..., "9"]</code>	Plain digit strings rendered as Phaser text objects
<code>stimulusType</code>	<code>"text"</code>	Phaser adds a text object, not an image
<code>interStimulusInterval</code>	500 ms	500 ms blank gap between trials
<code>debug</code>	true	Console logging ON in current code

 *Baseline now shows a green tint on the stimulus box for correct responses and a red tint for incorrect responses. The tint clears after 500 ms (the `interStimulusInterval`).*

7.2 practice.js — symbol practice

Practice introduces the symbol-digit mapping using the dedicated practice symbol set (frames p01 through p09 in the `symbolAtlas`). The symbol key grid is visible so the participant can look up the correct answer.

Parameter	Value in code	Meaning
<code>numTrials</code>	9	One trial per practice symbol
<code>stimuli</code>	<code>["p01", ..., "p09"]</code>	Frame names from the <code>symbolAtlas</code> for the practice set
<code>stimulusType</code>	<code>"image"</code>	Stimuli are images drawn from the <code>symbolAtlas</code>
<code>interStimulusInterval</code>	500 ms	500 ms blank gap between trials — new in this version
<code>randomize</code>	true	Shuffled order each time

randomType	"shuffle"	Fisher-Yates shuffle of the 9-item array
symbolSet	"p"	GameUI loads practice symbols from the atlas
debug	false	Console logging OFF in current code

NEW *practice.js now has `interStimulusInterval: 500` (was 0) and shows green/red tint feedback per response, matching baseline behaviour.*

7.3 run.js — the actual DSST

Run is the main test. It is time-limited to 60 seconds and can present up to 180 trials.

Parameter	Value in code	Meaning
numTrials	180	Upper ceiling; the timer ends the block first under normal conditions
numBlocks	1	Single block
blockDuration	60	Block ends after exactly 60 seconds
randomize	true	Symbols are shuffled before the block
randomType	"shuffle"	Fisher-Yates shuffle
stimulusType	"image"	Symbols from the symbolAtlas
debug	false	Console logging OFF — changed from true in previous version

NEW *After the 60-second block ends, run.js now shows a Message using the 'end_screen' text key ("This is the end of this task. The next task will start automatically."), then waits 5 seconds before calling `sendData()` to upload to Pavlovia.*

NEW *`sendData` is now imported and called. The data upload is triggered automatically — no manual wiring needed.*

8. Supporting Gameobjects

8.1 button.js

Button extends `Phaser.GameObjects.Container` and holds two children: a `Sprite` (the button image, toggling between `frameUp` and `frameDown` frames on press) and a `Text` label. Press animation moves child objects down by `pressOffset` pixels and resets on release. After release, the callback function is called if one was provided.

Button constructor parameters: `scene`, `x`, `y`, `texture` (atlas key), `frameUp`, `frameDown`, `labelText`, `labelSize` (relative to sprite height), `labelColor`, `active` (enables interaction immediately), `callback`. Unchanged in this version.

8.2 countdown.js

Countdown shows a 3-2-1 countdown in the center of the screen using a Phaser repeating timer (1000 ms interval, 3 repetitions). It emits a 'complete' event after the counter reaches 0. Unchanged in this version.

8.3 feedbackHandler.js — NEW

NEW *feedbackHandler.js is a new gameobject added in this version.*

FeedbackHandler shows a small popup overlay after each trial response. It is instantiated in baseline.js and practice.js (though the popup is currently displayed via a direct stimulus box tint rather than the popup UI — see below).

Constructor parameters:

- scene — the scene in which the popup is rendered
- x, y — canvas position of the popup center (default 0, 0)
- displayDuration (ms) — how long the popup stays visible (default 1000 ms)
- animate (boolean) — if true, the popup scales in with a Back.easeOut tween (default false)

show({ response, message }) method:

- Creates a rounded rectangle popup with a response icon (from guiAtlas, frame name = response) and a text label.
- If animate: true, tweens scale from 0 to 1. Otherwise shows immediately.
- Emits a 'complete' event after displayDuration milliseconds.

i *In the current code, baseline.js and practice.js use a simpler inline approach: they set a green (#00ff12) or red (#ff2c2c) tint directly on scene.stimulusBox and clear it after stimulusInterval ms. The FeedbackHandler popup calls are commented out. To enable the full popup, uncomment this.feedback.show(...) and remove the tint lines.*

8.4 gameInstructions.js

GameInstructions manages the animated overlay shown before each task starts. It reads the instruction text from the cache, then splits the string by comma to get individual instruction segments. Each segment maps to one instructionStep defined in the scene's configInstructions() method.

i *Because instruction text is split by comma, commas inside instruction strings will break the text into separate steps. Avoid commas in the text JSON instruction strings, or change the delimiter in gameInstructions.js line 79 if you need them.*

8.5 message.js

Message handles the DialogBox visual. It fades the camera out, then renders a text_box image with word-wrapped message text and a Button below. In this version it is also used by run.js to display the end_screen after the run completes.

8.6 sendData.js — now active

NEW *sendData is now imported in run.js and called automatically after the end screen is shown. In previous versions it was never called.*

sendData() builds a jsPsych timeline with a Pavlovian init and finish step:

- pavlovian_init — initialises the connection to Pavlovian using PARTICIPANT_ID.
- pavlovian_finish — uploads all jsPsych.data to the Pavlovian server and posts a 'surveyDone' message to the Pavlovian iframe.

The data written by TrialHandler and GameUI (trial index, stimulus, response, RT, correctness, timestamps, and all device metadata from boot.js) is what gets uploaded.

9. Instruction Text and Localisation

All participant-facing text lives in assets/text/NED_text.json (Dutch) or assets/text/ENG_text.json (English). The file is loaded in boot.js based on the LANG global.

***NEW** ENG_text.json is new in this version. It contains English translations of all keys. The end_screen and next_button keys are also new in both locale files.*

Key	Used by	Purpose
start_message	DialogBox (Start → Baseline)	Introductory message; introduces the 'crack the safe' narrative
practice_message	DialogBox (Baseline → Practice)	Explains that digits are now replaced by symbols
run_message	DialogBox (Practice → Run)	Final prompt: answer as many as possible in 60 seconds
baseline_instruction	GameInstructions in baseline.js	Two comma-separated segments: digit appears at top / press matching button
practice_instruction	GameInstructions in practice.js	Three comma-separated segments: symbol appears / find it in the key / press the button
end_screen	Message in run.js (NEW)	Shown after run completes: 'This is the end of this task. The next task will start automatically.'
next_button	DialogBox (Baseline → Practice) (NEW)	Label for the 'next' button on the practice_message dialog

To translate or change any text, edit only the JSON file. No JavaScript changes are needed. For a new language, copy the file, translate the values, and update the load path in boot.js. The language can be switched at runtime via the ?lang= URL parameter.

10. Configuration Quick-Reference for Students

10.1 Change trial count, block count, or time limit

File: src/scenes/run.js (or practice.js / baseline.js), in the PARAMETERS FOR TRIAL section at the top of create().

- More trials: increase numTrials.
- Multiple blocks: increase numBlocks. Remove blockDuration (or set to 0) to end blocks by trial count rather than time.
- Longer test: increase blockDuration (seconds). Set to 0 to disable the timer entirely.

10.2 Change stimulus timing

- stimulusDuration (ms) — how long a stimulus stays on screen before disappearing automatically. Default 0 = wait for button press.
- interStimulusInterval (ms) — blank gap between stimulus disappearing and next one appearing. Now 500 ms in baseline and practice; 0 in run.
- delayBeforeStart (ms) — pause between end of countdown and first stimulus. Currently 500 ms in all task scenes.

10.3 Disable randomization

File: src/scenes/practice.js or run.js, in the TrialHandler constructor. Set randomize: false. Stimuli will appear in the exact order of the stimuli array, repeated to fill numTrials.

10.4 Fix the symbol set and key order (disable day/hour rotation)

File: src/scenes/run.js.

- Remove the this.setStimuli() call from create().
- Hardcode directly: this.symbolSet = 's01'; this.symbolOrder = [1,2,3,4,5,6,7,8,9]; then build this.stimuli manually.

10.5 Change colors and fonts

File: assets/css/style.css. Edit the --primary-color, --secondary-color, --text-dark-color, or --text-light-color variables. Replace .ttf files in assets/fonts/ and update the @font-face src lines to use a different typeface.

10.6 Change instruction or end screen text

File: assets/text/NED_text.json (or ENG_text.json). Edit the string for any key. Use \n for line breaks in DialogBox messages. For GameInstructions text (baseline_instruction, practice_instruction): each comma-separated segment is one animated instruction step. The number of segments must match the number of steps in configInstructions(). The new end_screen key controls the message shown after the run completes.

10.7 Change language

Pass ?lang=en or ?lang=nl in the URL. This selects ENG_text.json or NED_text.json respectively. Default is nl.

 English is now supported out of the box via ENG_text.json. No code changes needed — just change the URL parameter.

10.8 Enable debug mode

Set debug: true in the TrialHandler constructor. The browser DevTools console will log each trial index, the stimulus key shown, and each button press. Note: debug is true in baseline.js and false in practice.js and run.js in the current code.

10.9 Enable the FeedbackHandler popup (instead of tint)

In baseline.js and practice.js: uncomment the this.feedback lines and remove the setTint / clearTint lines. The FeedbackHandler will show a popup with the correct/incorrect response icon for displayDuration ms.

10.10 Data collection

Data collection is now fully wired up and requires no extra steps. Each trial row in the CSV contains:

- Device metadata: fps, pixel ratio, OS, browser language, jsPsych version, Phaser version
- Session metadata: participant_id, beepnum, testday
- Trial data: trial.index, trial.UTCdate, trial.type, stim.type, stim.onset
- Response data: response.type, response.onset, response.RT, stim.key, response.key, response.correct

Data is uploaded to Pavlovia automatically after the end screen. CSV files from sessions are stored in the data/ folder.

11. Adding a New Symbol Set

All symbols are stored in sprite atlases in assets/textures/. Frame naming convention: first two characters are the set code (e.g. s01, p), followed by a two-digit number (01 through 09). So s0301 is symbol 1 of set 3.

To add a new symbol set (example: 's06'):

- Design 9 new symbol images and export as individual PNGs.
- Use a sprite atlas packer (TexturePacker, Shoebox, or gdx-texture-packer) to combine them into one atlas PNG and JSON descriptor. Name the frames s0601 through s0609.
- Save the atlas at three resolutions (1x, 2x, 3x) or add as separate files and register them in boot.js.
- If using separate files, add a new this.load.atlas() call in boot.js preload().
- In run.js setStimuli(), add the new set to the symbolSets array and add a time window entry in setPerHour to assign it to a time slot.

12. Common Mistakes and Tips

Problem	Likely cause	Solution
Symbols don't appear / show as blank	Atlas frame name in stimuli array does not match actual frame name in the JSON	Open the atlas .json file and check exact frame names — they are case-sensitive.
Task never ends / hangs after countdown	blockDuration: 0 with numTrials set very high, or playerResponse event never emitted	Set blockDuration > 0 for a time-limited block, or check recordResponse(true) is called after countdown.
Buttons don't respond to taps	recordResponse(true) not called, or called before countdown 'complete' event fires	Ensure gameUI.recordResponse(true) is inside the countdown.event.once('complete',...) callback.
Instruction animation skips or shows wrong text	Instruction text has unexpected commas, or textIndex count does not match segments	Count the commas in the text JSON for that key. Each comma creates one extra segment.
Black screen on load	Browser blocking ES module imports from file:// (no server running)	Always run via Live Server or python -m http.server. Never open index.html from the file system.
Landscape mode freezes the app	Intentional behaviour in main.js orientation check	By design — the task is portrait-only. On desktop, narrow the browser window.
Wrong symbol key layout	symbolOrder not matching expected mapping, or setStimuli() not called	Log GameUI debug output to see frame names and their expected digit keys.
Data not uploaded after the run	Pavlovia experiment is in Piloting mode, or PARTICIPANT_ID is missing from URL	Ensure the experiment is set to Running in Pavlovia and the ?participant= URL parameter is set.
Wrong language shown	?lang= parameter missing or wrong value	Add ?lang=en or ?lang=nl to the URL. Default is nl.

13. Glossary

Term	Definition
Scene	A Phaser concept: one self-contained screen or state of the app. Phaser runs one scene at a time (except Border, which runs persistently).
Sprite atlas / Texture atlas	A single large image file containing many small images, with a JSON file mapping each frame name to its pixel coordinates.
Frame	One individual image within a sprite atlas, identified by a string name (e.g. 's0101', 'button_up').

Trial	A single stimulus presentation plus one participant response.
Block	A set of trials. Can end by reaching numTrials, or by a blockDuration timer.
TrialHandler	The JavaScript class that manages the trial and block loop. Lives in src/gameobjects/trialHandler.js.
GameUI	The class that builds the symbol-key reference grid and the 3x3 numpad.
FeedbackHandler	NEW — A gameobject that displays a popup overlay after each response, showing whether it was correct or incorrect.
DPR (Device Pixel Ratio)	How many physical display pixels correspond to one CSS pixel. Used to load 1x, 2x, or 3x asset files.
playerResponse event	A Phaser scene event emitted when any numpad button is released. TrialHandler listens for it to advance to the next trial.
Pavlovia	An online platform for hosting psychology experiments with server-side data storage. sendData.js uploads to it via jsPsych.
Fisher-Yates shuffle	A standard algorithm for producing a uniformly random permutation of an array. Used by TrialHandler when randomType: 'shuffle'.
active: true (Phaser scene)	A Phaser Scene option that causes the scene to start running automatically at app launch. Used by Border.
LANG / PARTICIPANT_ID / BEEPNUM / TESTDAY	NEW — Global JavaScript constants set from URL parameters in index.html. Used to configure language, identify participants, and tag data rows.

14. File Edit Cheat-Sheet

What you want to change	File to open	Where inside the file
Trial count, block count, time limit	src/scenes/run.js or practice.js	Top of create() — the PARAMETERS FOR TRIAL comment block
Stimulus timing (duration, ISI, delay)	src/scenes/run.js or practice.js	TrialHandler constructor call and this.delayBeforeStart
Symbol set and day/hour rotation	src/scenes/run.js	setStimuli() method — orderPerDay and setPerHour arrays
Instruction and message text	assets/text/NED_text.json or ENG_text.json	The JSON key-value pairs (remember: commas split instruction steps)

Language selection	URL ?lang=nl en parameter	No code change needed — set via URL at runtime
Colors and fonts	assets/css/style.css	The :root CSS custom properties
Canvas size and scene order	src/main.js	The Phaser config object (width, height, scene array)
Asset loading (new atlases, images)	src/scenes/boot.js	The preload() method — this.load.atlas() / this.load.image()
Symbol-key layout logic	src/gameobjects/gameUI.js	The symbolOrder parameter (passed in from run.js / practice.js)
Button appearance / press frames	assets/textures/atlas_gui_*.png and *.json	Replace or edit the key_up and key_down frames in the atlas
End screen message	assets/text/NED_text.json or ENG_text.json	Edit the 'end_screen' key value
Feedback style (tint vs popup)	src/scenes/baseline.js or practice.js	Comment/uncomment feedback lines in the playerResponse handler
Instruction animation steps	src/scenes/baseline.js or practice.js	configInstructions() method — the instructionSteps array
Session parameters (participant, beepnum, testday)	URL parameters	Pass ?participant=ID&beepnum=N&testday=N in the URL

End of document

DSST Project Documentation | Updated June 2026